

專題競賽報告組別：資訊應用組 C4

系統名稱：無人機火災辨識系統

指導老師：鄭哲斌 老師

成員：

姓名	學號	修課班級	座號
陳宇柔	410630809	資管四 B	031
曾紘杰	410630249	資管四 B	013
謝長恩	410630080	資管四 B	009
張楚霖	410631138	資管四 B	042
呂紘燁	410631161	資管四 B	043
朱婕瑜	411637050	資管四 B	060

113 學年度

目錄

壹、前言.....	3
一、動機與目的.....	3
二、系統目標.....	5
三、系統 LOGO.....	6
貳、系統功能簡介.....	7
一、系統架構圖.....	7
二、使用者操作介面.....	9
三、無人機飛行、辨識.....	13
四、資料庫.....	14
五、Line Bot.....	15
六、Power BI.....	17
參、系統使用對象.....	18
肆、系統特色.....	19
伍、系統設計概要.....	20
一、系統流程圖.....	20
二、專案架構.....	21
三、處理流程.....	25
三、實際作品流程.....	42
陸、系統開發環境及工具.....	50
柒、結論及未來發展.....	55
一、結語.....	55
二、未來展望.....	55
捌、參考資料.....	56
玖、專題月進度表.....	57
拾、消防隊參訪.....	59
拾壹、分工表.....	62

壹、前言

一、動機與目的

當前的國際消防安全形勢正面臨嚴峻的挑戰，主要表現在以下幾個方面：

1. 極端天氣：氣候變遷帶來的極端天氣，使火災更加難以預測和控制。
2. 森林火災頻發：全球範圍內的森林火災規模和面積不斷創下新高，給各國的消防救援能力帶來巨大壓力。
3. 公共安全威脅：火災不僅對人們的生命和財產構成威脅，還可能對社會經濟造成深遠影響。
4. 城市化影響：隨著城市化進程的加快，火災管理面臨更多複雜的挑戰，傳統的火災應對措施常常難以適應。
5. 無人機系統的興起：無人機系統因其靈活、高效的特性，在火災管理中展現了重要潛力。

(1)火災偵測：無人機能夠快速、準確地偵測火源，提供實時影像和數據。

(2)應急反應：在火災發生時，無人機可以迅速部署，幫助救援人員進行現場勘察和指揮調度。

(3)災後評估：無人機能夠在火災後進行災區的詳細勘察，提供精確的損害評估數據。

(4)預防措施：無人機可以用於火災預警和巡邏，及早發現潛在火災隱患。

(5)消防安全：無人機的先進技術可以提升消防工作效率，提高整體消防安全水平。

隨著技術的不斷進步和無人機系統的普及，未來的火災應對工作將變得更加高效、科學和安全。因此，研究和推廣無人機系統在火災管理中的應用，具有重要的實際意義和社會價值。

二、系統目標

1. 增強火災應急反應

利用無人機進行火場探查，搭配火焰與顏色辨識，幫助消防員快速且安全的確認火勢狀況。

2. 強化災後評估與救援

紀錄每次救援情況與時間，搭配 Power BI 利用圖表化的形式快速統計與分析消防狀況。

3. 優化火災管理和措施

利用無人機的高機動性與快速反應能力，提供關鍵之空中影像及實時數據，進而協助消防隊員做出準確判斷。

4. 提升消防人員的安全性

使用無人機高空協助滅火，幫助消防人員快速控制火災現場，縮減消防人力並進一步降低消防隊員之滅火風險。

5. 強化災後評估與救援

利用無人機進行災後探勘，並評估需要派遣多少人力，減少災後探勘人力之浪費。

三、系統 LOGO



上圖為此系統之 LOGO，以下是設計理念：

1. 消防頭盔：頭盔是消防員的重要象徵，代表了保護與勇氣。它作為消防行業的象徵，並代表救火與拯救生命的使命。
2. 火焰：火焰的圖像在頭盔上方蔓延，強烈代表火災以及消防的核心工作——控制和撲滅火災。火焰的動感設計強化了緊急與危險的氛圍，也提醒著火勢的無情。
3. 無人機元素：頭盔下方的圓形造型是以無人機的鏡頭作為抽象概念。想表達無人機技術在現代消防中越來越重要，能夠提供火場的空中視野，幫助消防隊更精確地判斷火勢。
4. 顏色對比：火焰的紅色、橙色與頭盔的深藍及灰色形成強烈的色彩對比，既表現了火的熱烈與危險，也突顯了消防員冷靜、鎮定的特質。

整體而言，此設計結合傳統的消防元素（如頭盔和火焰）與現代技術（如無人機），表達了消防工作的核心價值：保護生命、控制火勢，同時亦表達無人機在消防行動中為日漸重要的角色。

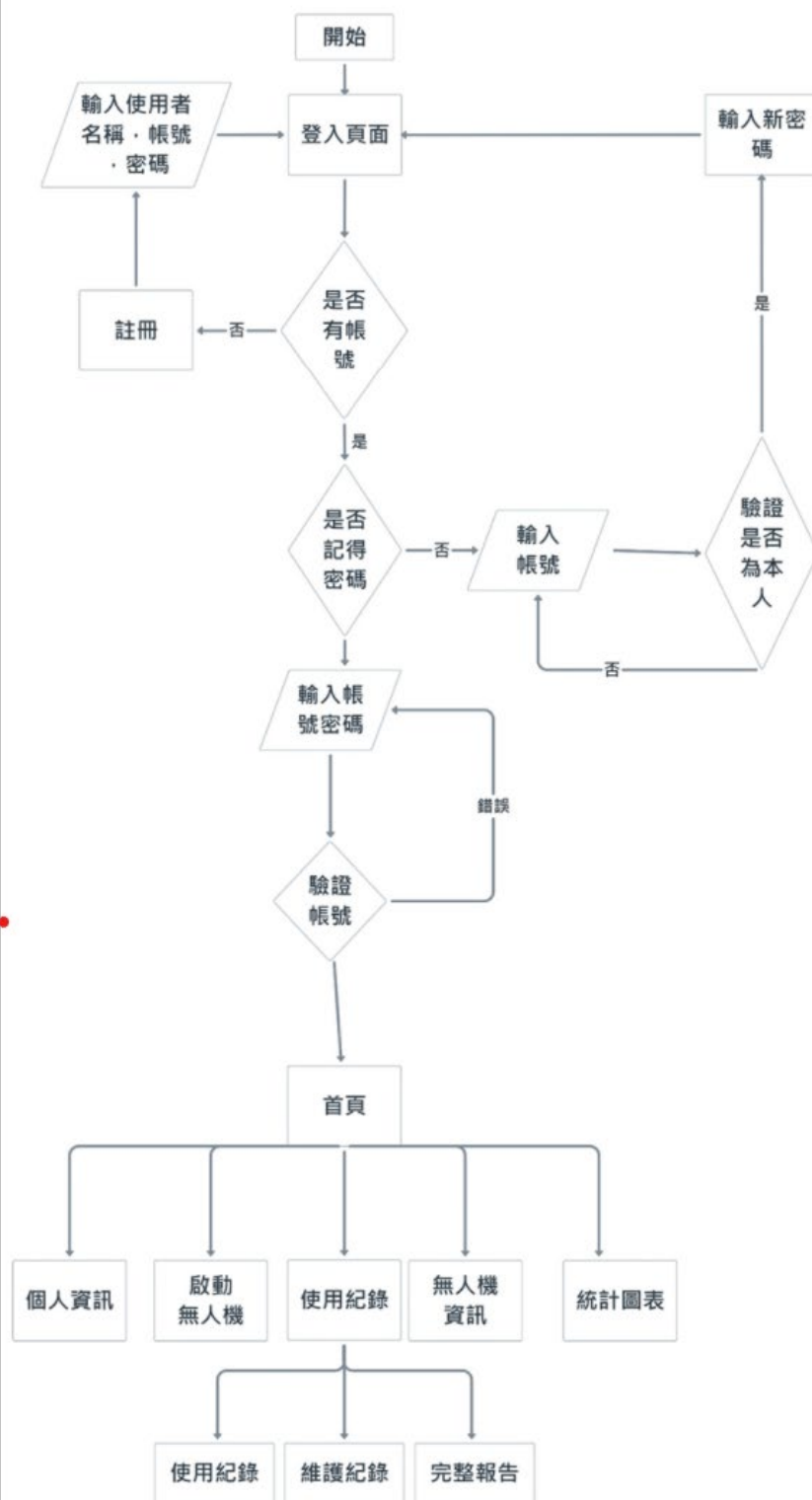
貳、系統功能簡介

一、系統架構圖

此系統為無人機火災辨識系統，進到介面後會先看到我們的登入頁面，接著會請消防隊員輸入帳號、密碼，第一次使用會需要註冊帳號，之後就可以再登入頁面直接進行登入的動作。

若不幸忘記密碼，亦有設計忘記密碼功能，輸入當初註冊之姓名及帳號，認證成功即可設定新密碼。登入成功便會自動跳轉至首頁，首頁部分有 5 個按鈕，分別是左邊區塊的「啟動無人機」，以及右邊部分的「使用紀錄」、「統計圖表」、「無人機資訊」、「個人資訊」。

而在「使用紀錄」的部分，亦可看到更詳細的使用報告及維護紀錄，並生成完整使用報告，希望消防隊員能藉由此系統更有效率的儲存資訊，及搜索/整理所有資料。



二、使用者操作介面

此為提供給消防隊員使用之管理系統，在介面設計的部分特別採用精簡易閱讀的頁面，方便消防隊員在使用上能以最快的速度，找到需要的資訊。

在主色彩設計上採用淺灰色，灰色被視為一種中性的顏色，在視覺上具有平衡與安定的效果。

由於灰色介於黑色與白色之間，並不會像鮮豔的顏色那樣刺激眼睛，也不會像深色那樣沉重。這使得灰色成為一種較為溫和、低調且不會分散注意力的顏色。

在輔色上選用水藍色。我們認為消防的核心理念通常與「火」和「熱」相關，傳統上聯想到紅色、橙色等溫暖和強烈的顏色。而水藍色則是冷色調，象徵「水」、「冷靜」和「平靜」。這兩種概念可以通過以下方式結合：

1. 水與火的平衡：水藍色象徵著消防人員使用水滅火、控制火勢的過程，這是一種冷靜、克制火焰的力量。藍色和紅色的對比可以體現消防工作中「冷靜應對危機」的精神，並強調水作為滅火的主要工具。

2. 保護與安全感：水藍色具有平和、安定的感覺，能傳達出消防部門在保障人民安全時的可靠和鎮定。這種顏色可以用來突顯消防救援中的專業和理性，與「火」的緊急感形成視覺對比，讓人感到安全。

此系統內含無人機使用紀錄、統計圖表、無人機資訊、消防員個人資訊。

1. 登入頁面

登入頁面包含輸入帳號、密碼、忘記密碼、建立新帳號等功能。



無人機消防
管理系統

帳號 root

密碼 123

登入

忘記密碼?

建立新帳號



忘記密碼

使用者帳號

姓名

新密碼

確認修改



建立帳號

姓名

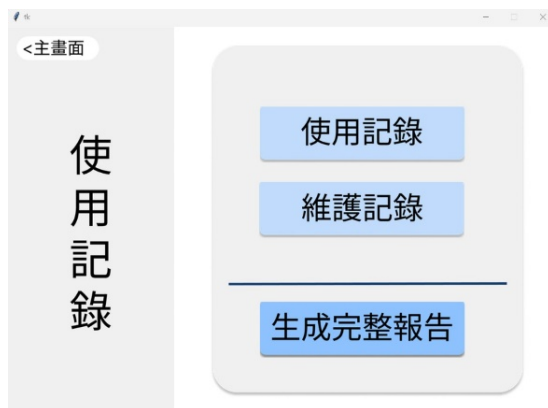
使用者帳號

使用者密碼

建立帳戶

2. 主頁面

主頁面包含啟動無人機、使用紀錄、統計圖表、無人機資訊、個人資訊。



The '使用記錄' (Usage Record) page displays a table of drone usage records. The table has five columns: '任務ID' (Task ID), '無人機ID' (Drone ID), '任務類型' (Task Type), '開始時間' (Start Time), and '結束時間' (End Time). Below the table are two buttons: '新增資料' (Add Data) and '刪除資料' (Delete Data).

任務ID	無人機ID	任務類型	開始時間	結束時間
3	1	救災	2024-05-02 08:20:00	2024-05-02
4	1	救災	2024-05-02 08:30:00	2024-05-02
5	1	救災	2024-05-02 08:40:00	2024-05-02
6	1	救災	2024-05-02 08:50:00	2024-05-02
7	1	救災	2024-05-02 09:00:00	2024-05-02
8	1	救災	2024-05-02 09:10:00	2024-05-02
9	1	救災	2024-05-02 09:20:00	2024-05-02
10	1	救災	2024-05-02 09:30:00	2024-05-02
11	1	救災	2024-05-02 09:40:00	2024-05-02
12	1	救災	2024-05-02 09:50:00	2024-05-02
13	1	救災	2024-05-02 10:00:00	2024-05-02

The '個人資訊' (Personal Information) page has a sidebar with the title '<主畫面' and '個人資訊'. The main content area features a large person icon and a form with three input fields: '姓名' (Name), '使用者帳號' (User Account), and '編號' (ID Number). Below the form is a '登出' (Logout) button.

The '無人機資訊' (Drone Information) page has a sidebar with the title '<主畫面' and '無人機資訊'. The main content area displays a table of drone information.

無人機ID	型號	製造日期
1	drone1	2024-05-01
2	drone2	2024-05-20
3	drone3	2024-05-25

3. 統計圖表

我們將資料庫資訊結合 Python，繪製出許多不同的統計圖表，使消防隊員能一目了然找到需要的資訊，並查看統計資料。



三、無人機飛行、辨識

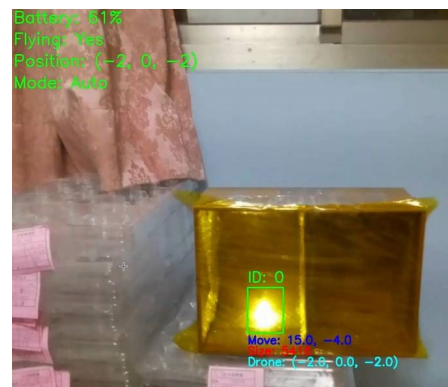
1. 辨識：

使用 YOLOv8 訓練火焰辨識模型，結合顏色辨識系統輔助，增加火焰辨識之精確度。實時判斷火焰大小，並飛行至最大火源處撲滅火勢，待火焰撲滅後回到原始位置，並重新執行任務。

2. 實際 demo

利用紙箱+玻璃紙+燈泡，模擬大樓火災現場火勢情況，分別會有大區域(4 顆燈泡)/中區域(2 顆燈泡)/小區域(1 顆燈泡)，無人機首先進行辨識，畫面會出現 3 個紅框，表示辨識到 3 個火焰區域，無人機自指定位置起飛後會優先飛至最大火焰區域滅火，待火勢撲滅(燈光熄滅)，無人機會接著回到原始起飛位置，並重新執行辨識，並飛向當前火焰最大區域，之所以讓無人機重新回到原位辨識，是為防止在第一次滅火期間，其餘區域之火災嚴重情況有所改變。

若是遇到火焰區域相等之情況，則會以相對位置較近之區域先行滅火。



四、資料庫

以上辨識資料、使用紀錄等資料皆利用資料庫儲存，使消防隊員能保存數據，不怕遺失

```
mysql> show tables;
+-----+
| Tables_in_mydatabase |
+-----+
| drones                |
| maintenancerecords     |
| missions              |
| users                 |
+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM drones;
+-----+
| DroneID | Model   | ManufactureDate | MaxPayload | MaxFlightTime | MaxFlightDistance | Status |
+-----+
| 1       | drone1  | 2024-05-01      | 25.00      | 8.00          | 100.00            | Available |
| 2       | drone2  | 2024-05-20      | 35.00      | 8.00          | 100.00            | Available |
| 3       | drone3  | 2024-05-25      | 30.00      | 8.00          | 100.00            | Available |
+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM maintenancerecords;
+-----+
| RecordID | DroneID | MaintenanceDate | MaintenanceType | MaintenanceDetails | MaintenanceCost |
+-----+
| 1        | 1       | 2024-05-01      | 購買           | 購買無人機         | 1000.00         |
| 2        | 1       | 2024-05-08      | 保養           | 保養電機           | 200.00          |
| 7        | 1       | 2024-07-23      | 保養           | 清潔               | 120.00          |
+-----+
3 rows in set (0.00 sec)

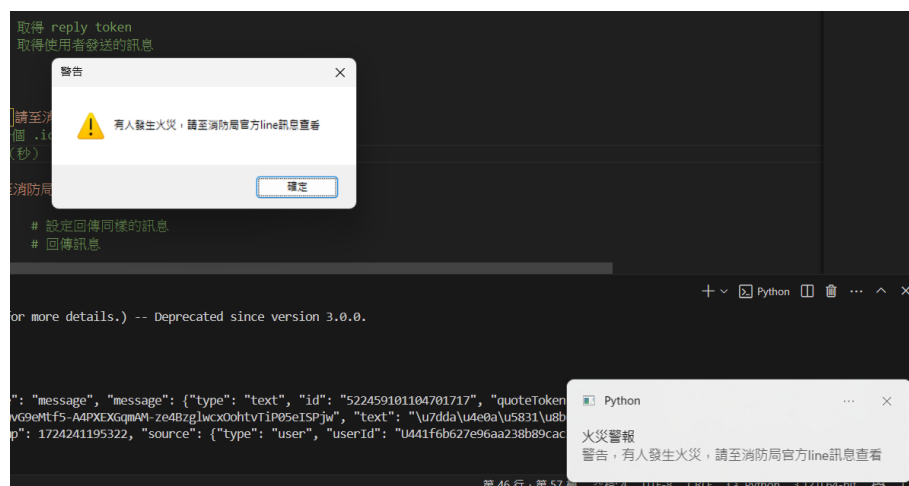
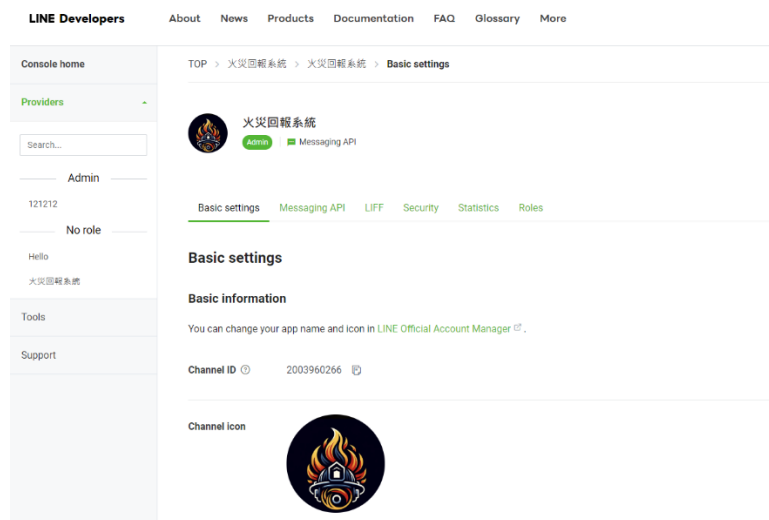
mysql>
mysql> SELECT * FROM missions;
+-----+
| MissionID | DroneID | MissionType | StartTime | EndTime | MissionLocation | Success | MaxHeight | MinHeight | totalFlight |
+-----+
| 3         | 1       | 救災       | 2024-05-02 08:28:00 | 2024-05-02 08:23:00 | Taipei     | 1       | 80       | 75       | 1          |
| 4         | 1       | 救災       | 2024-05-02 08:36:00 | 2024-05-02 08:34:30 | Taipei     | 1       | 65       | 35       | 1          |
| 5         | 1       | 救災       | 2024-05-02 08:40:00 | 2024-05-02 08:42:56 | Taipei     | 0       | 70       | 30       | 1          |
| 6         | 1       | 救災       | 2024-05-02 08:50:00 | 2024-05-02 08:55:20 | Taipei     | 1       | 50       | 45       | 1          |
| 7         | 1       | 救災       | 2024-05-02 09:00:00 | 2024-05-02 09:04:30 | Taipei     | 0       | 100      | 85       | 1          |
| 8         | 1       | 救災       | 2024-05-02 09:10:00 | 2024-05-02 09:12:30 | Taipei     | 1       | 120      | 90       | 1          |
| 9         | 1       | 救災       | 2024-05-02 09:26:00 | 2024-05-02 09:26:30 | Taipei     | 0       | 130      | 110      | 1          |
| 10        | 1       | 救災       | 2024-05-02 09:36:00 | 2024-05-02 09:37:20 | Taipei     | 1       | 70       | 64       | 1          |
| 11        | 1       | 救災       | 2024-05-02 09:48:00 | 2024-05-02 09:43:10 | Taipei     | 1       | 150      | 120      | 1          |
| 12        | 1       | 救災       | 2024-05-02 09:50:00 | 2024-05-02 09:52:00 | Taipei     | 0       | 140      | 123      | 1          |
| 13        | 1       | 救災       | 2024-05-02 10:00:00 | 2024-05-02 10:06:30 | Taipei     | 1       | 130      | 90       | 1          |
| 14        | 1       | 救災       | 2024-05-05 11:42:42 | 2024-05-05 11:44:44 | Taipei     | 1       | 50       | 45       | 1          |
| 18        | 1       | 救災       | 2024-05-13 12:17:32 | 2024-05-13 12:17:39 | Taipei     | 1       | 120      | 100      | 1          |
| 20        | 1       | 救災       | 2024-06-17 13:54:34 | 2024-06-17 13:58:40 | Taipei     | 1       | 110      | 50       | 1          |
| 21        | 1       | 救災       | 2024-07-24 15:18:02 | 2024-07-24 15:18:12 | Taipei     | 1       | 120      | 110      | 2          |
+-----+
15 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM users;
+-----+
| UserID | Username | Password | Name | CreationTime | LastLoginTime |
+-----+
| 1      | user    | 123     | aaa  | 2024-04-21 22:45:53 | 2024-08-07 20:57:25 |
| 2      | admin   | 123     | bbb  | 2024-07-30 15:49:42 | 2024-07-30 16:46:51 |
| 3      | root    | 123     | ccc  | 2024-07-30 15:54:12 | 2024-08-10 14:26:13 |
| 4      | gggg    | 123     | ddd  | 2024-07-30 17:45:19 | 2024-07-30 18:15:03 |
| 7      | paris   | 12345   | 鄭哲斌 | 2024-08-07 21:51:17 | 2024-08-07 21:51:26 |
+-----+
5 rows in set (0.00 sec)
```

五、Line Bot

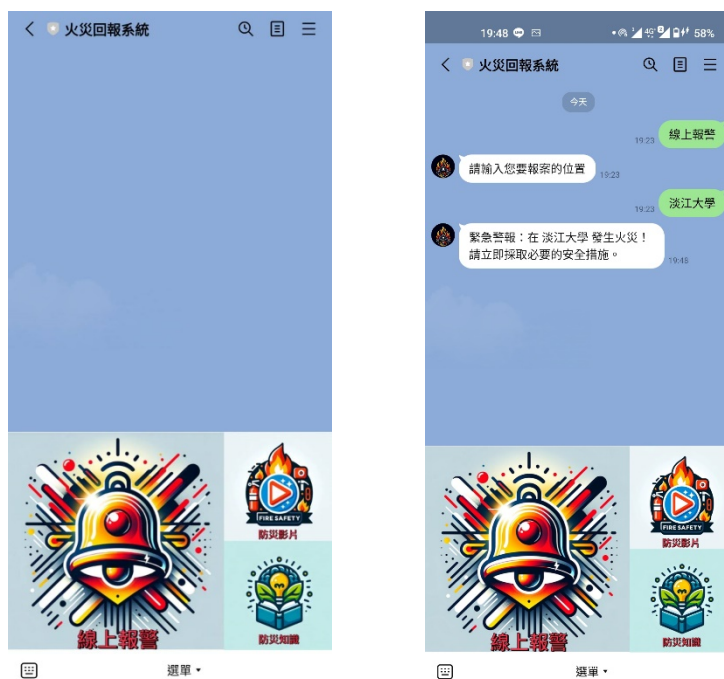
1. 管理端

使用 LINE Developers 建立官方帳號，名為「火災回報系統」，當接收到線上報警時電腦會跳出通知。



2. 使用者端

民眾可加入此官方帳號，並從此介面「線上報警」，也能進一步獲得發生火災地點。

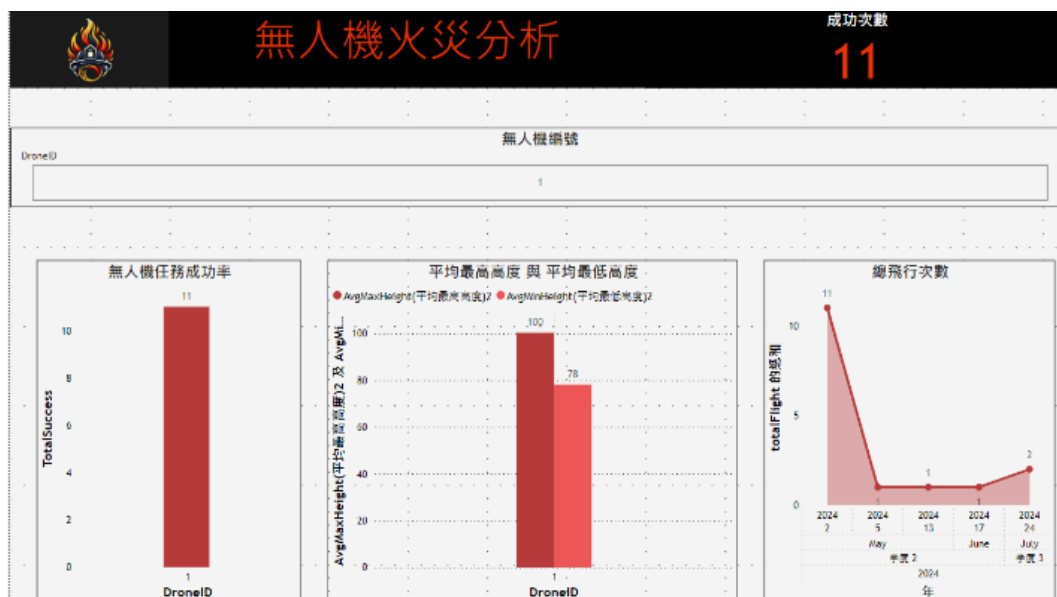
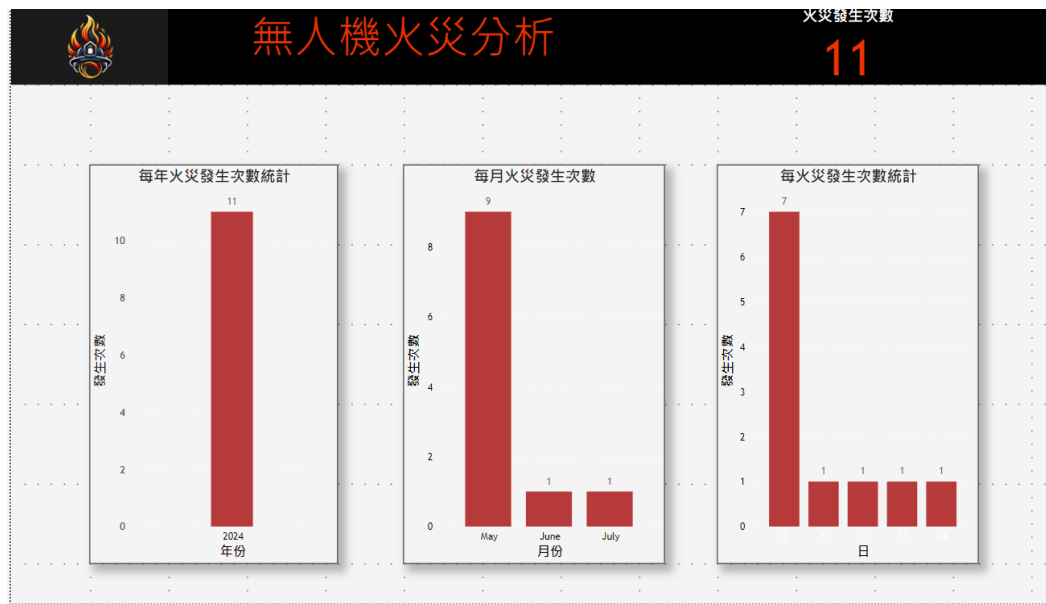


平時亦可從此介面，學習防災小知識及火災避難逃生要領。



六、Power BI

使用 Power BI 製作火災分析圖表，讓消防員能夠整合資訊，使救災資訊一目了然。我們運用了線上資料庫，從中獲取即時資料來更新畫面中的圖表，隨著無人機辨識到的可用資訊來增加圖表，有利於使用者更清楚明白了解火災歷史紀錄，而可獲得資訊有：時間區間成功次數、任務成功率、無人機飛行平均高度及總飛行時數。



參、系統使用對象

一、消防隊與緊急救援單位

利用無人機和此系統快速定位火源、評估火災範圍，特別適用於大型火災或難以進入的地區，協助制定滅火策略，並提高救援效率和安全性。

二、一般民眾

設計Line Bot，讓需要的使用者都能藉由加入Line好友使用此系統，加入後即能獲取火災預警、緊急聯絡資訊等服務。

肆、系統特色

一、火焰辨識

使用yolo v8進行模型訓練，配合顏色辨識系統，增加火焰辨識準確度。

二、消防輔助

利用無人機進行火場偵查，快速確認火災戰場，方便消防人員快速制定消防戰術並搭配火焰辨識系統進行半自動滅火。

三、報警系統

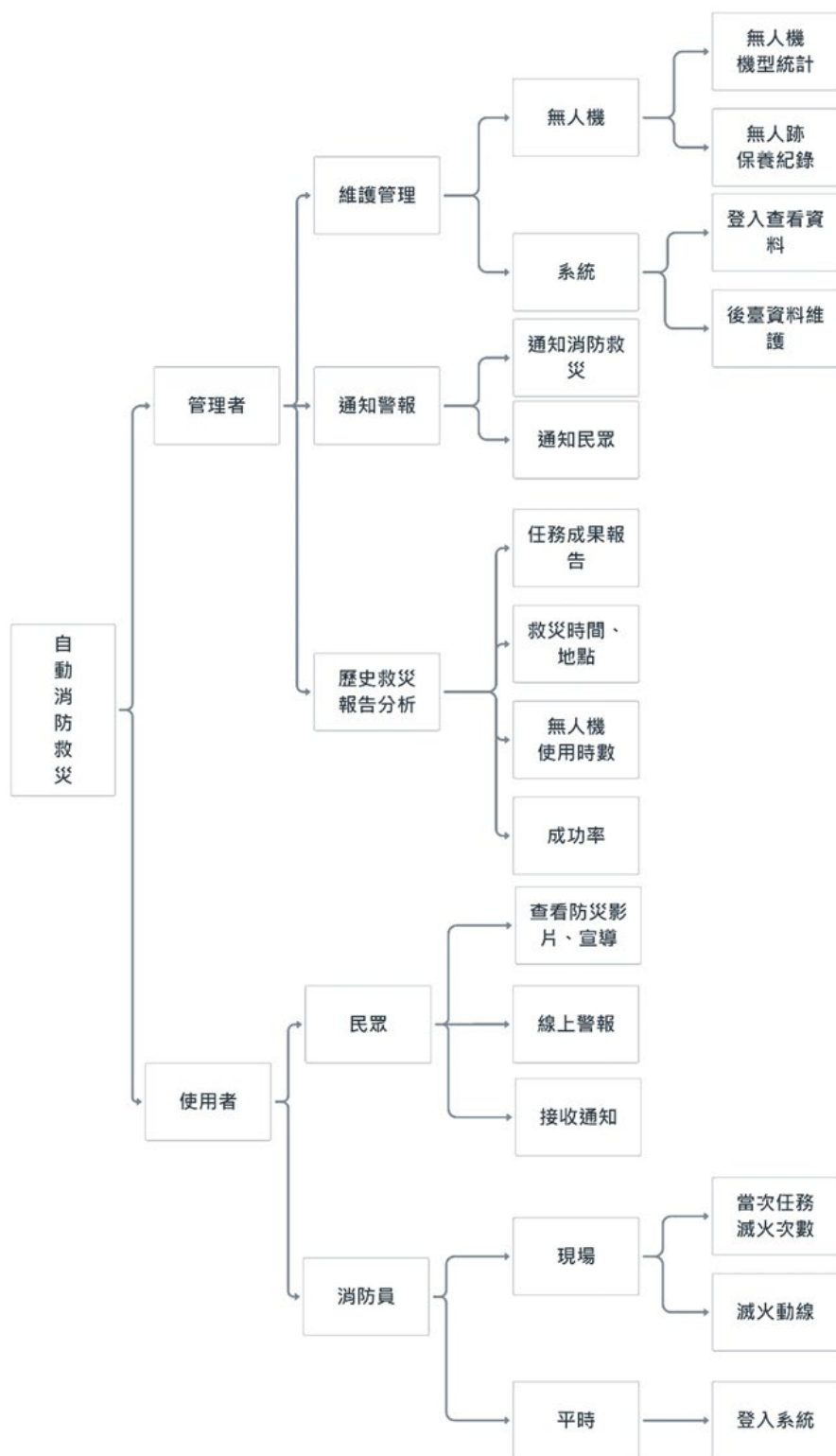
將各種資料存入資料庫並連結至power BI，利用視覺化資訊方便消防人員進行討論與戰術制定。

四、及時警報系統

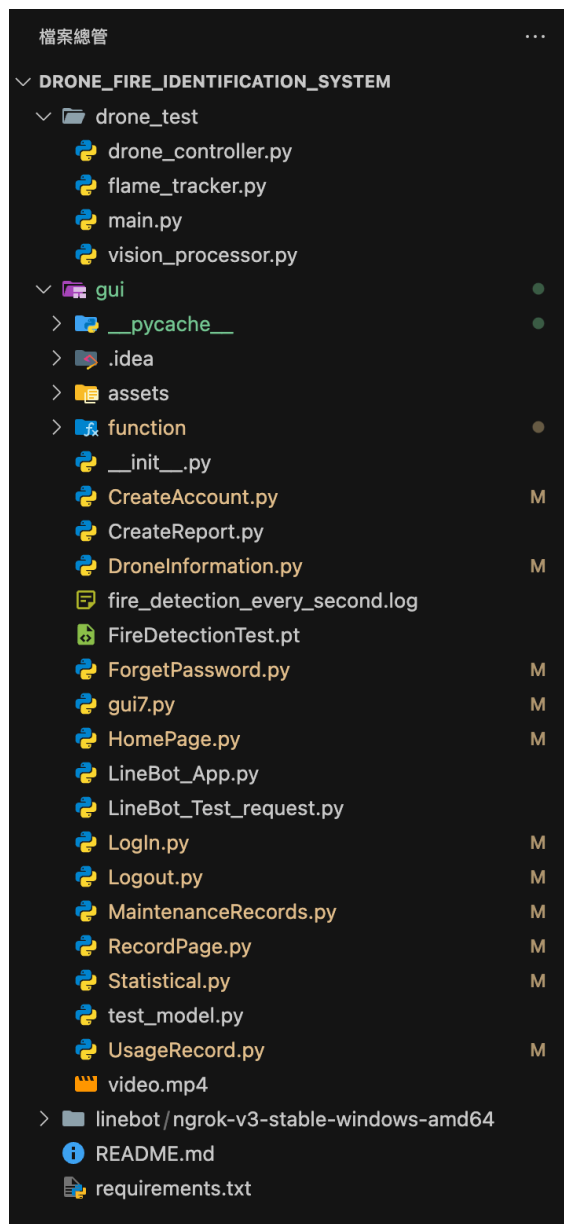
使用line帳號連結民眾端，發生火災可使用其進行報警，啟動無人機消防系統後通知所有用戶火災發生地，使用戶能及時了解火災資訊並遠離火災現場，增加消防反應時間。

伍、系統設計概要

一、系統流程圖



二、專案架構



專案名為 DRONE_FIRE_IDENTIFICATION_SYSTEM

每個目錄和檔案的作用：

1. drone_test 資料夾

存放無人機操作和火災辨識的程式碼，包含以下 Python 檔案：

drone_controller.py：控制無人機飛行。

flame_tracker.py：檢測和追蹤火焰位置，透過影像處理追蹤火源。

main.py：專案主入口，整合無人機控制和火焰檢測模組，負責啟動系統。

vision_processor.py：負責視覺資料的處理、圖像處理邏輯，具體包括色彩轉換、遮罩處理，形態學操作來清理噪音，並最終識別火焰。該檔案與 flame_tracker.py 配合使用，共同提供視覺資料作為火焰追蹤的基礎。

2. gui 資料夾 (ui 檔案在 gui，每一個畫面一個檔案，所需的對應函數程式在 \gui\function 裡，方便維護。)

圖形使用者介面部分包含以下資料夾：

(1).idea 資料夾

PyCharm IDE 產生，用來儲存專案的設定、配置等開發環境資訊。

(2) assets 資料夾

存放 Figma 製作的前端介面 png 檔。

(3) function 資料夾

寫 gui 的對應功能

(4) ui 檔案、Lint Bot、測試模型

CreateAccount.py：負責用戶註冊或建立帳號的功能。

CreateReport.py：是生成檢測報告的檔案。

DroneInformation.py：用來顯示或管理無人機資訊。

ForgetPassword.py：提供忘記密碼的重設功能。

HomePage.py：啟動無人機、開啟其他頁面。

LogIn.py 和 LogOut.py：分別負責用戶登入和登出的功能。

MaintenanceRecords.py：負責維護記錄的管理功能。

RecordPage.py：記錄無人機活動、火災檢測資料等。

Statistical.py：統計分析、顯示結果。

UsageRecord.py：負責使用記錄的管理功能。

LineBot_App.py 和 LineBot_Test_request.py：Line Bot 應用程式和測試請求檔案。

test_model.py：測試火災辨識模型。

3. fire_detection_every_second.log

日誌檔案，記錄了每秒的火災檢測結果。

4. FireDetectionTest.pt 檔案

PyTorch 模型檔案，用來進行火災檢測的模型。

5. linebot 資料夾

ngrok 的設定，是用來進行本地測試的。

6. requirements.txt

專案中所需的 Python 套件或依賴庫。

7. README.md：專案的說明文件。

三、處理流程

1. 辨識技術

(1) 訓練 YOLOv8 模型



用訓練出來的火焰辨識模型進行辨識並畫框標記，並實時判斷最大火焰的位置以及火焰中心相對於影像中的位置。

● 火焰與煙霧的辨識

YOLOv8 是一個即時物件偵測模型，對輸入的影片進行即時分析，並在畫面上標註出火焰和煙霧的位置。每一個偵測框會包含以下資訊：

座標框（Bounding Box）：顯示偵測到的火焰或煙霧範圍。

信心值 (Confidence)：模型對於該偵測結果的信心程度，範圍從 0 到 1。越接近 1 表示模型對該結果越有信心。

● 火焰追蹤與消失判斷

為了有效追蹤火焰，我們實作火焰追蹤邏輯，系統中每個火焰被當作一個物件進行追蹤：

火焰物件：每個火焰物件包含其座標框和火焰中心點座標。透過火焰的中心點來比較兩次偵測的火焰是否為同一個物件。

追蹤邏輯：比較兩個火焰中心點的距離，如果距離小於預設閾值（200 像素），我們就認為這是同一團火焰。每次偵測時，若發現新的火焰，則會將其加入追蹤列表。

火焰消失判斷：當某個火焰在影片中超過一定時間（5 秒）未被偵測到，系統會將該火焰標註為「消失」。

● 火焰大小與顏色標示

每個偵測到的火焰或煙霧範圍會根據其面積來分配不同顏色，這樣可以快速區分火焰的大小，規則如下：

小於 1000 像素面積：用綠色框顯示，表示小範圍火焰。

1000 到 5000 像素面積：用藍色框顯示，表示中等範圍火焰。

大於 5000 像素面積：用紅色框顯示，表示較大範圍火焰。

這樣的顏色區分可以直觀地顯示火焰的大小，協助快速辨識火災的嚴重程度。

- 火焰中心點與數據顯示

程式還會顯示火焰的平均中心點位置和一些統計資料：

火焰的平均中心點：程式會計算所有被追蹤火焰的中心點平均值，這樣可以估計整體火焰的分佈位置。

數據顯示：程式會在畫面上顯示總火焰數量、當前偵測到的火焰數量、最大的火焰座標框位置，以及火焰偵測的平均信心值。

- 每秒紀錄火焰偵測結果

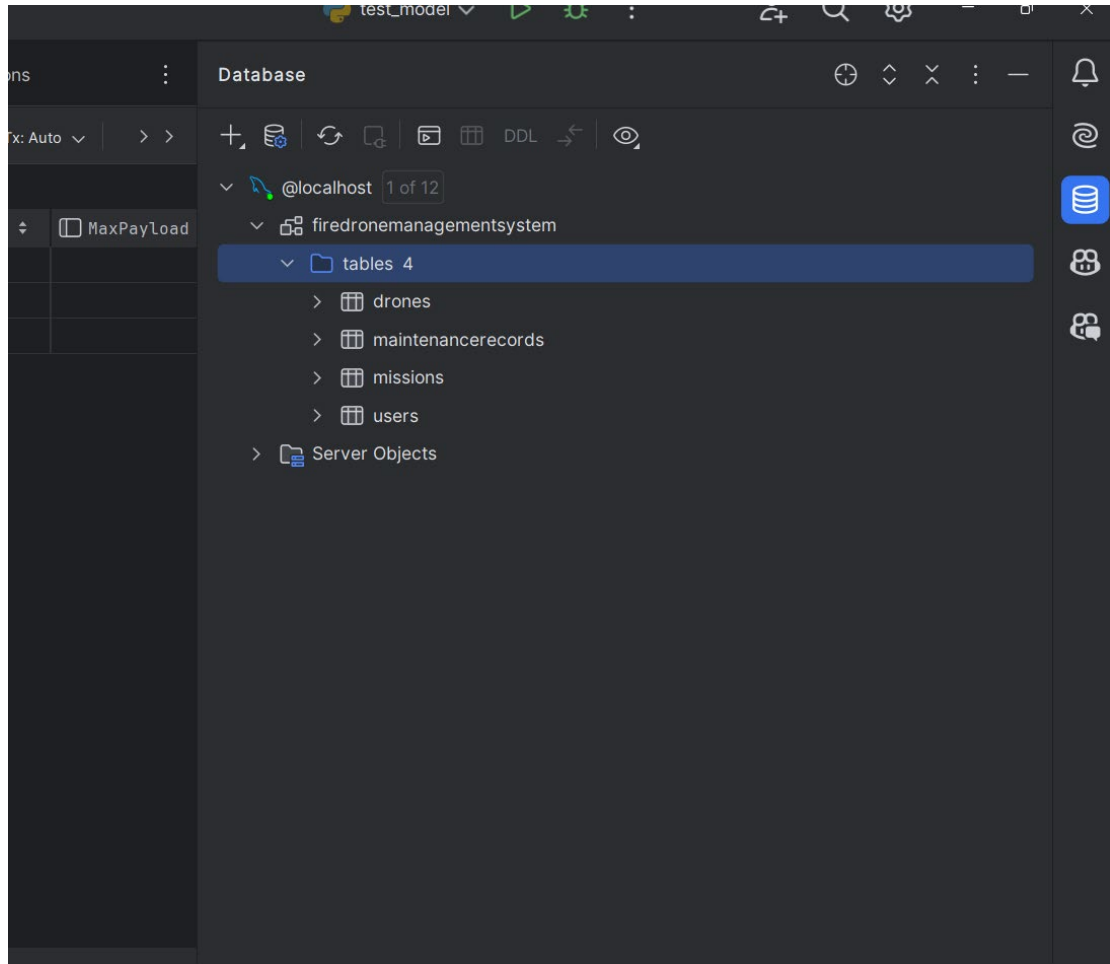
程式每秒會將偵測到的火焰數據（如總偵測火焰數量、當前火焰數量、最大火焰框位置、信心值、火焰中心點）記錄到日誌文件中，便於後續分析火災的演變情況。

(2)訓練用圖集

<https://universe.roboflow.com/sina-rashidi/fire-classification-ztrfn/dataset/1>

2. 資料庫

後端使用 PyCharm 來管理資料庫。並在學校虛擬機架設遠端資料庫以連接 Power BI 及後端。



架設遠端資料庫時遇到連接問題。

推斷過程：

ping IP 沒問題，確定網路通順。

telnet port 5656、3389 都可以，但 3306 不行。

確認 MySQL 正在監聽 3306 端口。

```
[tkuim-sd@tkuimsd-virtual-machine:~$ sudo netstat -tuln | grep 3306
[[sudo] password for tkuim-sd:
tcp        0      0 0.0.0.0:33060 0.0.0.0:*        LISTEN
```

MySQL 配置文件調整為 bind-address = 0.0.0.0，允許所有 IP 連

接。

bind-address	= 0.0.0.0
mysqlx-bind-address	= 0.0.0.0

確認 MySQL 用戶權限 tkuim-sd 的 host 是「%」，表示允許來自所有 IP 連接。

```
mysql> SELECT user, host FROM mysql.user WHERE user = 'tkuim-sd';
+-----+-----+
| user  | host |
+-----+-----+
| tkuim-sd | %    |
+-----+-----+
1 row in set (0.00 sec)
```

排除以上，最後推測是 3306 端口(MySQL 預設端口)未開放的問題。

解決方法：把端口換到 3307。

3. 無人機的選擇

製作專題的過程中，我們共購買了 3 台無人機，分別是：

- (1) CoDrone EDU：第一台測試之無人機，但因此機型沒有鏡頭，不符合我們需求。
- (2) Tello：經過第一台無人機的考量後，換成購買 Tello 來使用，但在使用上發現稍有不足。
- (3) Tello RoboMaster TT：這是我們最後選擇使用的無人機，Tello RoboMaster TT 是一款由 DJI 和 RoboMaster 合作推出的教育無人機，專門設計用來讓學生學習編程和開發，是 Tello 的升級版。它保留了 Tello 無人機的基本功能，並進一步增加了教育和創新應用的功能。

4. 無人機操控（介紹部分重要方法）

(1) 控制結構與邏輯

初始化與連接（initialize 方法）：通過建立 Tello 物件來連接無人機，並啟動視訊串流功能。這樣無人機可以開始接收命令並返回即時影像。

連接控制（connect 方法）：當連接成功後，無人機會不斷檢查其狀態，例如電池電量，並保持與無人機的連接。

自動與手動模式切換（toggle_mode 方法）：有手動和自動控制兩種模式。

(2) 主要功能與邏輯

保持命令（send_heartbeat_command 方法）：無人機會不定期發送保持命令來確保其懸停在空中，防止無人機因為沒有接收到指令而觸發自動降落。

調整位置與移動（adjust_position 方法）：根據火焰的位置及大小，無人機會自動計算移動的速度與方向，例如前後、上下移動和旋轉方向，這些參數是通過固定的速度選項來選擇的，從而保證飛行的穩定性。

自動模式 (move_to_flame 方法)：無人機根據偵測到的火焰資訊，計算火焰相對於無人機的距離與位置，進行相應的追蹤、移動。

手動控制 (_handle_manual_control 方法)：使用者可以通過鍵盤等外部命令來操作無人機的起飛、降落、方向調整等操作。

(3) 火焰追蹤與影像處理

火焰選擇方法 (select_target_flame 方法)：無人機支援三種火焰選擇方式，包括選擇最大的火焰、最近的火焰和最早發現的火焰。這些選擇邏輯能確保無人機始終能追蹤到最合適的火源，防止無效移動。

```
222 def select_target_flame(self, selection_method, frame_width, frame_height):
223     all_flames = {**self.yolo_flames, **self.color_flames}
224
225     if not all_flames:
226         return None, None
227
228     if selection_method == "largest":
229         target_id = max(all_flames, key=lambda x: all_flames[x]['smoothed_size'])
230     elif selection_method == "nearest":
231         target_id = min(all_flames, key=lambda x:
232             abs(all_flames[x]['box'][0] + all_flames[x]['box'][2] / 2 - frame_width / 2) +
233             abs(all_flames[x]['box'][1] + all_flames[x]['box'][3] / 2 - frame_height / 2))
234     elif selection_method == "oldest":
235         target_id = min(all_flames, key=lambda x: all_flames[x]['first_detected'])
236     else:
237         return None, None
238
239     # 優先選擇 YOLO 檢測的火焰
240     yolo_flames = list(self.yolo_flames.keys())
241     if yolo_flames:
242         if target_id not in yolo_flames:
243             target_id = yolo_flames[0]
244
245     return target_id, all_flames[target_id]
```

YOLO 影像辨識 (process_frame 方法)：進行畫面處理，從影像中辨識出火焰的具體位置。(可參考 p. 41)

顏色檢測 (detect_flame_color 方法)：除了 YOLO 模型外，系統利用 HSV 色彩範圍來檢測火焰，通過顏色偵測進行輔助判斷。

```
6 class VisionProcessor:
206     def detect_flame_color(self, frame):
207         hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
208         mask = cv2.inRange(hsv, self.lower_flame, self.upper_flame)
209
210         # 測試
211         # 定義兩個HSV範圍
212         light_source_lower = np.array([0, 0, 240])
213         light_source_upper = np.array([100, 10, 255])
214
215         extended_light_lower = np.array([0, 220, 160])
216         extended_light_upper = np.array([20, 255, 255])
217
218         # 創建兩個遮罩
219         mask_light_source = cv2.inRange(hsv, light_source_lower, light_source_upper)
220         mask_extended_light = cv2.inRange(hsv, extended_light_lower, extended_light_upper)
221
222         # 合併兩個遮罩
223         mask = cv2.bitwise_or(mask_light_source, mask_extended_light)
224
225         # 測試碼結束
226
227         # 進行形態學操作以減少噪聲
228         kernel = np.ones((5, 5), np.uint8)
229         mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
230         mask = cv2.dilate(mask, kernel, iterations=2)
231
232         # 找到輪廓
233         contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
234
235         color_detections = []
236         for contour in contours:
237             if cv2.contourArea(contour) > 100: # 過濾小區域
238                 x, y, w, h = cv2.boundingRect(contour)
239                 color_detections.append([x, y, x + w, y + h, 0.5]) # 使用固定的置信度 0.5
240
241         return color_detections
```

(4)系統安全性與緊急狀態處理

緊急降落 (emergency_land 方法)：無人機配備了緊急降落功能，當發生錯誤或突發情況時，可以立即觸發降落保護無人機和周圍環境的安全。

電池監控 (get_battery 方法)：無人機會定時監控電池狀況，確保電量在安全範圍內運行，並在低電量時及時通知使用者。

(5)整合與運行

多線程處理：整個系統利用多線程 (threading) 來確保影像處理、無人機控制和其他任務能夠並行執行。例如，影像處理與無人機

控制是分別在獨立的線程中進行的，這樣可以保證無人機的實時反應不會被影像處理延遲影響。

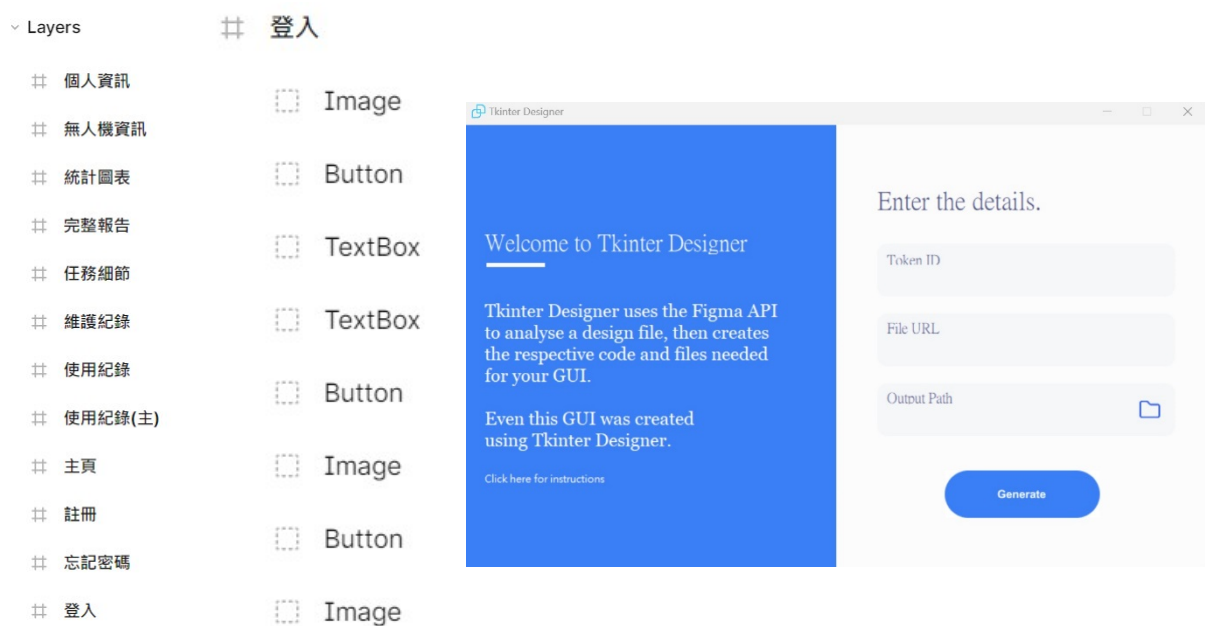
輸入 (`_process_user_input` 方法)：使用者可以通過鍵盤輸入進行操作，例如按 `t` 控制起飛降落、`m` 切換模式、`x` 執行緊急降落等。

5. 使用者介面

使用 Figma 設計使用者介面，先將區塊及框線、功能鍵拉好後，並結合成群組以及更改為 python 可辨識之名稱。

左圖為使用者介面之所有頁面，中圖則是以登入頁面為例。

Button 代表會轉換成按鈕，而 TextBox 則是會轉換為可打字之文字框。右圖則是 Tkinter Designer 頁面，輸入從 figma 獲取之金鑰，以及該檔案的網址，就可以將頁面輸出成.py 檔儲存。



6. Line Bot

建立 Line Bot 帳號：

- 註冊 LINE Developer 帳號：到 LINE Developer Console

註冊並建立 Channel。

- 取得 Channel ID、Secret 和 Access Token：這些資訊用

來串接 API，與後端進行溝通。



Basic settings **Messaging API** LIFF Security Statistics Roles

Messaging API settings

Bot information

Bot basic ID @563iznjh 

QR code



後端：

當偵測系統發現火災時，會發送 POST 請求到 /fire-alert，包含火災的位置信息（如 "淡江大學"）。

Flask 伺服器接收到火災警報，使用 LINE Messaging API 向指定

的 LINE 使用者發送火災訊息。

如果用戶在 LINE 發送訊息（如 "線上報警"），伺服器會顯示桌面通知和彈出式警告框提醒火災訊息。

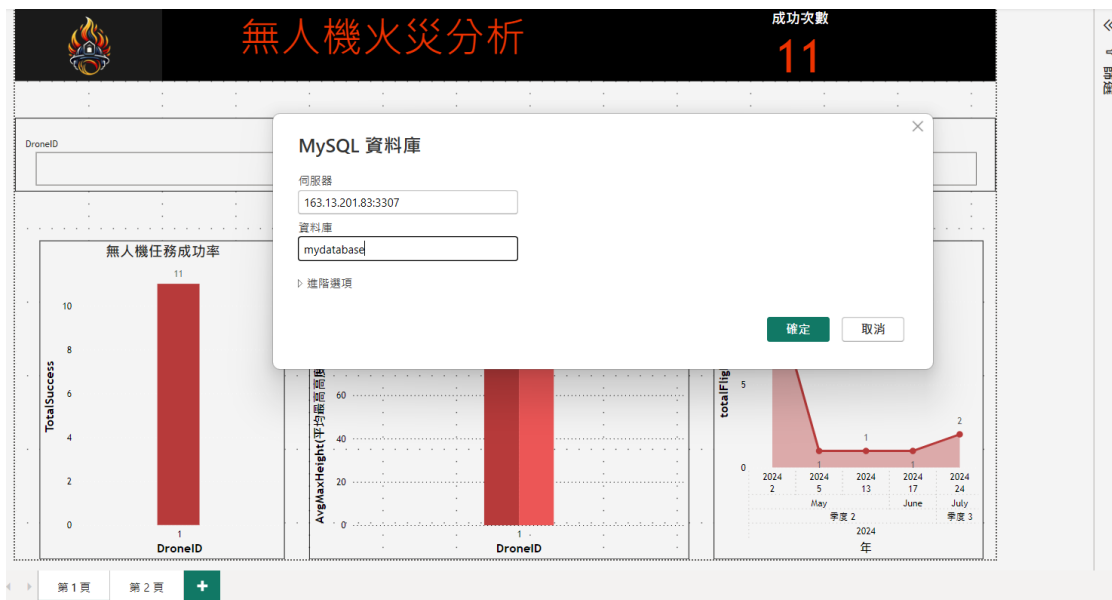
同時，系統可以打印回應的狀態碼和伺服器回應內容，以確保資料正確送達。

結論：這樣系統可以有效整合火災偵測與即時通訊系統，為用戶提供即時的火災警報通知。

7. Power BI

(1) 資料處理

- 連接資料庫：將資料庫中的數據匯入 Power BI，這些數據包含無人機的使用紀錄、火災報告、任務細節等。
- 即時資料更新：使用 Power BI 的線上資料庫連接功能，確保圖表能根據無人機辨識的即時數據進行動態更新。



(2)數據建模

- 清理數據：使用 Power BI 的查詢編輯器清理和轉換數據，

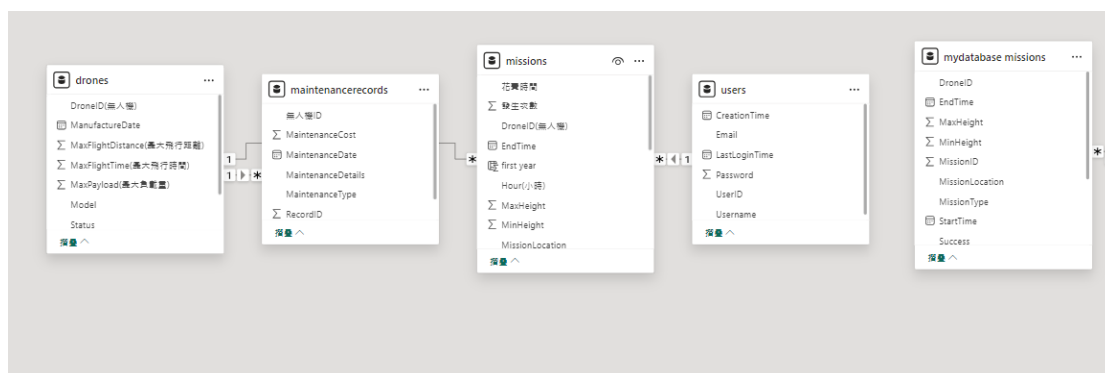
例如去除重複值、處理遺漏數據等，確保數據一致性和準確性。

	DroneID	MissionType	StartTime	EndTime	MissionLocation	Success
1	9	1 救災	2024/5/2 上午 08:20:00	2024/5/2 上午 08:23:00	Taipei	TRUE
2	4	1 救災	2024/5/2 上午 08:30:00	2024/5/2 上午 08:34:30	Taipei	TRUE
3	5	1 救災	2024/5/2 上午 08:40:00	2024/5/2 上午 08:42:56	Taipei	FALSE
4	6	1 救災	2024/5/2 上午 08:50:00	2024/5/2 上午 08:55:20	Taipei	TRUE
5	7	1 救災	2024/5/2 上午 09:00:00	2024/5/2 上午 09:04:30	Taipei	FALSE
6	8	1 救災	2024/5/2 上午 09:10:00	2024/5/2 上午 09:12:30	Taipei	TRUE
7	9	1 救災	2024/5/2 上午 09:20:00	2024/5/2 上午 09:26:30	Taipei	FALSE
8	10	1 救災	2024/5/2 上午 09:30:00	2024/5/2 上午 09:37:20	Taipei	TRUE
9	11	1 救災	2024/5/2 上午 09:40:00	2024/5/2 上午 09:43:10	Taipei	TRUE
10	12	1 救災	2024/5/2 上午 09:50:00	2024/5/2 上午 09:52:00	Taipei	FALSE
11	13	1 救災	2024/5/2 上午 10:00:00	2024/5/2 上午 10:06:30	Taipei	TRUE
12	14	1 救災	2024/5/5 上午 11:42:42	2024/5/5 上午 11:44:44	Taipei	TRUE
13	18	1 救災	2024/5/13 下午 12:17:32	2024/5/13 下午 12:17:39	Taipei	TRUE
14	20	1 救災	2024/6/17 下午 01:54:34	2024/6/17 下午 01:58:40	Taipei	TRUE
15	21	1 救災	2024/7/24 下午 03:18:02	2024/7/24 下午 03:18:12	Taipei	TRUE

- 創建關聯模型：將不同資料表（如無人機資訊、火災紀錄、

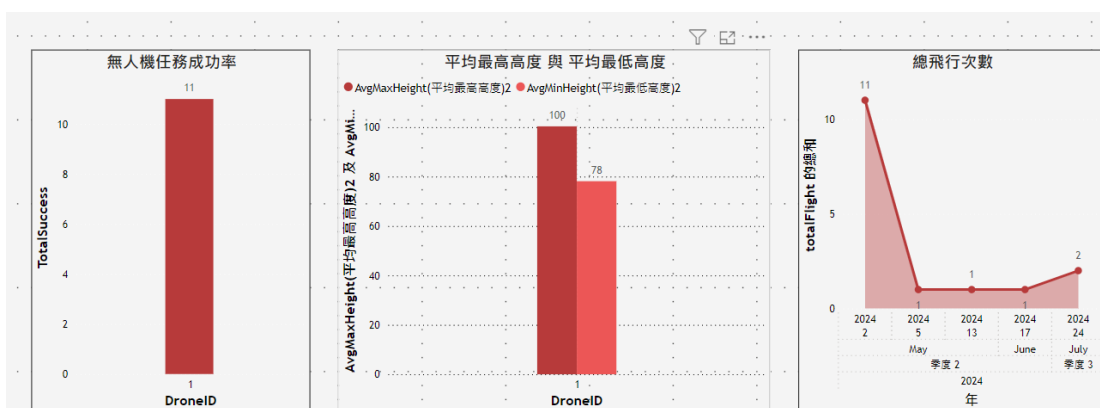
任務紀錄等）進行關聯，讓各個數據來源能在同一模型中交互使

用。



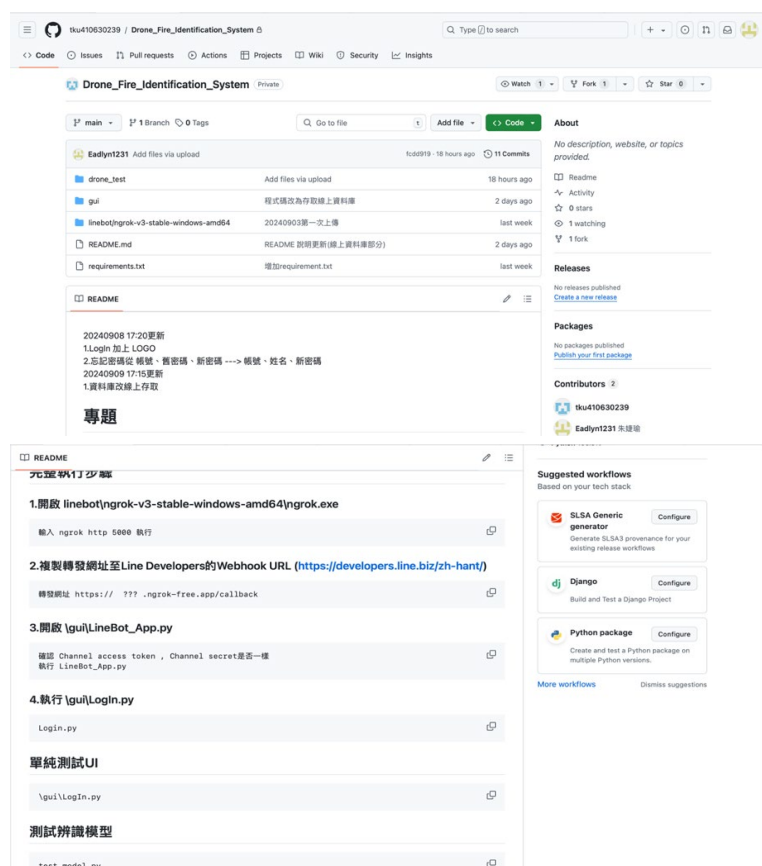
(3)建立視覺化報表

- 火災分析圖表：根據無人機回傳的數據，生成不同類型的圖表，幫助消防隊員了解火災的歷史紀錄、成功救援次數、無人機飛行高度和總飛行時數等數據。



8. Git 版本控制

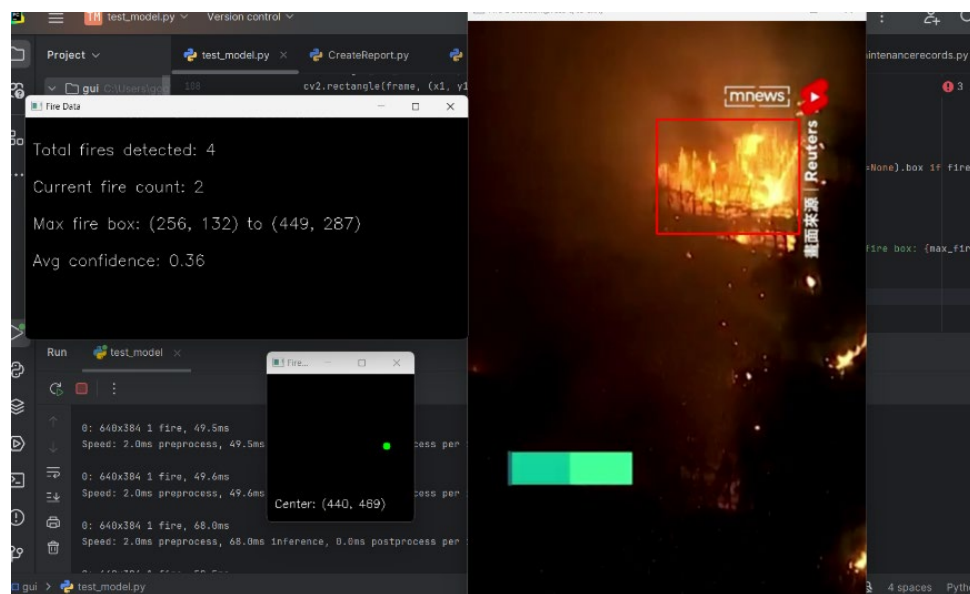
使用 Git 可以進行版本控制、追蹤歷史紀錄，增加多人協作的便利性等。能夠記錄每次的代碼變更、清楚看到編輯者提交的評論（comment），並保存所有歷史版本。大幅提升了開發靈活性、穩定性及可維護性。



三、實際作品流程

1. 核心辨識技術

(1) YOLO 模型



用訓練出來的火焰辨識模型進行辨識並畫框標記，並實時判斷最大火焰的位置以及火焰中心相對於影像中的位置。

(2) 無人機結合 YOLO 影像辨識

首先，在 VisionProcessor class 中，YOLO 模型初始化。

```
24 def initialize(self):
25     """
26     初始化 YOLO 模型
27     """
28     try:
29         self.yolo_model = YOLO(self.model_path)
30         print("YOLO 模型已初始化")
31         # 測試模型
32         dummy_frame = np.zeros((self.frame_height, self.frame_width, 3), dtype=np.uint8)
33         self.yolo_model(dummy_frame)
34         print("YOLO 模型測試成功")
35     except Exception as e:
36         print(f"初始化 YOLO 模型時發生錯誤: {str(e)}")
37         raise
```

其次，進入 YOLO 處理過程，每一幀影像通過模型進行處理，提取出火焰的邊界框和置信度，並返回檢測結果。

`process_frame` 方法處理來自無人機攝像頭的影像幀，並將這些幀送入 YOLO 模型。`_extract_detections` 方法會提取 YOLO 偵測到的火焰位置與置信度。

```
drone_test > vision_processor.py
6 class VisionProcessor:
38
39 def process_frame(self, frame):
40     """
41     處理單個視頻幀
42     :param frame: 輸入的視頻幀
43     :return: 處理後的幀和檢測到的目標
44     """
45     try:
46         if frame is None:
47             return None, [], []
48
49         frame = cv2.resize(frame, (self.frame_width, self.frame_height))
50
51         # YOLO 檢測
52         results = self.yolo_model(frame)
53         yolo_detections = self._extract_detections(results)
54
55         # 顏色檢測
56         color_detections = self.detect_flame_color(frame)
57
58         # 應用非最大抑制
59         all_detections = np.array(yolo_detections + color_detections)
60         if len(all_detections) > 0:
61             scores = all_detections[:, 4]
62             keep = self._non_max_suppression(all_detections[:, :4], scores, 0.3)
63             all_detections = all_detections[keep]
64
65         # 區分 YOLO 檢測和顏色檢測
66         yolo_detections = [d for d in all_detections if d[4] > 0.5]
67         color_detections = [d for d in all_detections if d[4] <= 0.5]
68
69         frame_with_detections = self._draw_detections(frame, yolo_detections, color_detections)
70
71         print(f"YOLO 檢測結果: {len(yolo_detections)}")
72         print(f"顏色檢測結果: {len(color_detections)}")
73
74         return frame_with_detections, yolo_detections, color_detections
75
76 except Exception as e:
77     print(f"處理幀時發生錯誤: {str(e)}")
78     return frame, []
```

再者，將 YOLO 偵測結果與無人機結合，在 `FireFightingDrone` class 中，`_update_drone_state` 方法負責處理無人機的狀態更新、取得 YOLO 偵測的火焰資訊，並將其與無人機控制相結合，根據火焰的位置信息來決定無人機的移動方向與速度。

```

drone_test > main.py
8 class FireFightingDrone:
90
91     def _update_drone_state(self):
92         try:
93             with self.lock:
94                 if self.frame is None:
95                     print("警告：沒有可用的幀")
96                     return
97
98                 frame = self.frame.copy()
99
100             if self.drone_controller.is_vision_enabled:
101                 frame, yolo_detections, color_detections = self.vision_processor.process_frame(frame)
102                 yolo_flames, color_flames = self.flame_tracker.update(yolo_detections, color_detections,
103                                                                     self.drone_controller.position)
104
105                 # 確保 yolo_flames 和 color_flames 不是 None
106                 yolo_flames = yolo_flames or {}
107                 color_flames = color_flames or {}
108
109                 all_flames = {**yolo_flames, **color_flames}
110                 yolo_flame_ids = set(yolo_flames.keys())
111
112                 frame = self.vision_processor.draw_flames(frame, all_flames, self.drone_controller.target_flame_id,
113                                                         yolo_flame_ids)
114
115                 print(f"YOLO 檢測到的火焰數量: {len(yolo_flames)}")
116                 print(f"顏色檢測到的火焰數量: {len(color_flames)}")
117
118                 if self.drone_controller.auto_mode and self.drone_controller.is_flying:
119                     self._handle_auto_mode(all_flames)
120                 elif not self.drone_controller.auto_mode:
121                     self._handle_manual_control()
122
123                 drone_info = self._get_drone_info()
124                 frame = self.vision_processor.display_info(frame, drone_info)
125
126                 cv2.imshow("Fire Fighting Drone", frame)
127         except Exception as e:
128             print(f"更新無人機狀態時發生錯誤: {str(e)}")
129             import traceback
130             traceback.print_exc()

```

最後，自動飛行模式下的火焰追蹤，無人機會根據 YOLO 偵測結果來移動。`_handle_auto_mode` 方法負責根據偵測到的火焰來自動調整無人機的位置。檢查是否有偵測到火焰，然後自動讓無人機移動到火焰附近。如果沒有偵測到火焰，無人機將保持懸停。

```

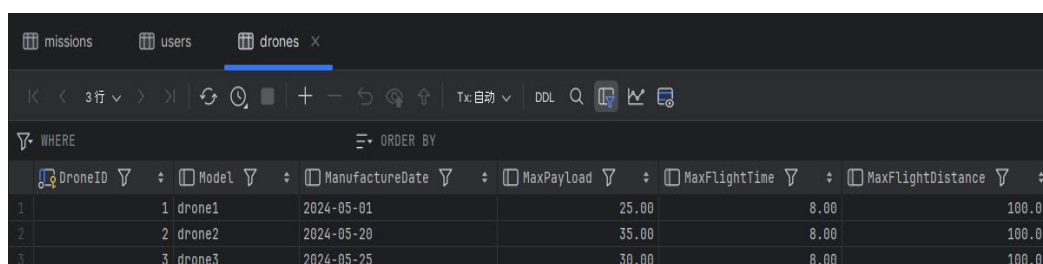
drone_test > main.py
8 class FireFightingDrone:
132
133     def _handle_auto_mode(self, all_flames):
134         """處理自動飛行模式"""
135         print("Entering _handle_auto_mode")
136         try:
137             if all_flames:
138                 target_id, target_flame = self.flame_tracker.select_target_flame(
139                     self.drone_controller.flame_selection_method,
140                     self.vision_processor.frame_width,
141                     self.vision_processor.frame_height)
142                 if target_flame:
143                     self.drone_controller.target_flame_id = target_id
144                     self.drone_controller.move_to_flame(target_flame,
145                                                         self.vision_processor.frame_width,
146                                                         self.vision_processor.frame_height)
147             else:
148                 self.drone_controller.adjust_position(0, 0, 0, 0) # 保持懸停
149             self.drone_controller.adjust_position(0, 0, 0, 0) # 保持懸停
150         except Exception as e:
151             print(f"Error in _handle_auto_mode: {str(e)}")
152             raise
153         print("Exiting _handle_auto_mode")

```

2. 無人機實際 demo 過程 參考 p.13

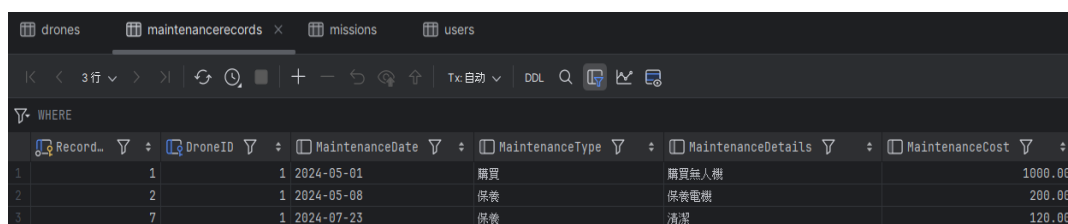
3. 資料庫

Drones 表記錄無人機 ID(DroneID)，型號(Model)，製造日期(ManufactureDate)，最大載重(MaxPayload)，最大飛行時間(MaxFlightTime)，最大飛行距離(MaxFlightDistance)



	DroneID	Model	ManufactureDate	MaxPayload	MaxFlightTime	MaxFlightDistance
1	1	drone1	2024-05-01	25.00	8.00	100.00
2	2	drone2	2024-05-20	35.00	8.00	100.00
3	3	drone3	2024-05-25	30.00	8.00	100.00

Maintancerecords 表記錄記錄 ID(RecordID)，無人機 ID (DroneID)，維護日期(MaintenanceDate)，維護類型(MaintenanceType)，維護詳情(MaintenanceDetails)，維護費用(MaintenanceCost)



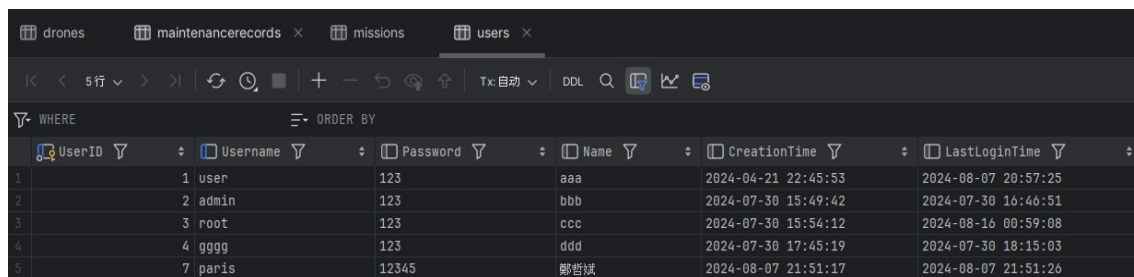
	RecordID	DroneID	MaintenanceDate	MaintenanceType	MaintenanceDetails	MaintenanceCost
1	1	1	2024-05-01	購買	購買無人機	1000.00
2	2	1	2024-05-08	保養	保養電池	200.00
3	7	1	2024-07-23	保養	清潔	120.00

Missions 表記錄任務 ID(MissionID), 無人機 ID(DroneID), 任務類型(MissionType), 開始時間(StartTime), 結束時間(EndTime), 任務地點(MissionLocation), 是否成功(1:成功, 0:失敗)(Success), 最大高度(MaxHeight), 最低高度(MinHeight), 架次(totalFlight)

MissionID	DroneID	MissionType	StartTime	EndTime	MissionLocation	Success
1	3	1 救災	2024-05-02 08:20:00	2024-05-02 08:23:00	Taipei	1
2	4	1 救災	2024-05-02 08:30:00	2024-05-02 08:34:30	Taipei	1
3	5	1 救災	2024-05-02 08:40:00	2024-05-02 08:42:56	Taipei	0
4	6	1 救災	2024-05-02 08:50:00	2024-05-02 08:55:20	Taipei	1
5	7	1 救災	2024-05-02 09:00:00	2024-05-02 09:04:30	Taipei	0
6	8	1 救災	2024-05-02 09:10:00	2024-05-02 09:12:30	Taipei	1
7	9	1 救災	2024-05-02 09:20:00	2024-05-02 09:26:30	Taipei	0
8	10	1 救災	2024-05-02 09:30:00	2024-05-02 09:37:20	Taipei	1
9	11	1 救災	2024-05-02 09:40:00	2024-05-02 09:43:10	Taipei	1
10	12	1 救災	2024-05-02 09:50:00	2024-05-02 09:52:00	Taipei	0
11	13	1 救災	2024-05-02 10:00:00	2024-05-02 10:06:30	Taipei	1
12	14	1 救災	2024-05-05 11:42:42	2024-05-05 11:44:44	Taipei	1
13	18	1 救災	2024-05-13 12:17:32	2024-05-13 12:17:39	Taipei	1
14	20	1 救災	2024-06-17 13:54:34	2024-06-17 13:58:40	Taipei	1
15	21	1 救災	2024-07-24 15:18:02	2024-07-24 15:18:12	Taipei	1

							MaxHeight	MinHeight	totalFlight
1	00	00	00	00	00	00	80	75	1
2	00	00	00	00	00	00	65	35	1
3	00	00	00	00	00	00	70	30	1
4	00	00	00	00	00	00	50	45	1
5	00	00	00	00	00	00	100	85	1
6	00	00	00	00	00	00	120	90	1
7	00	00	00	00	00	00	130	110	1
8	00	00	00	00	00	00	70	64	1
9	00	00	00	00	00	00	150	120	1
10	00	00	00	00	00	00	140	123	1
11	00	00	00	00	00	00	130	90	1
12	00	00	00	00	00	00	50	45	1
13	00	00	00	00	00	00	120	100	1
14	00	00	00	00	00	00	110	50	1
15	00	00	00	00	00	00	120	110	2

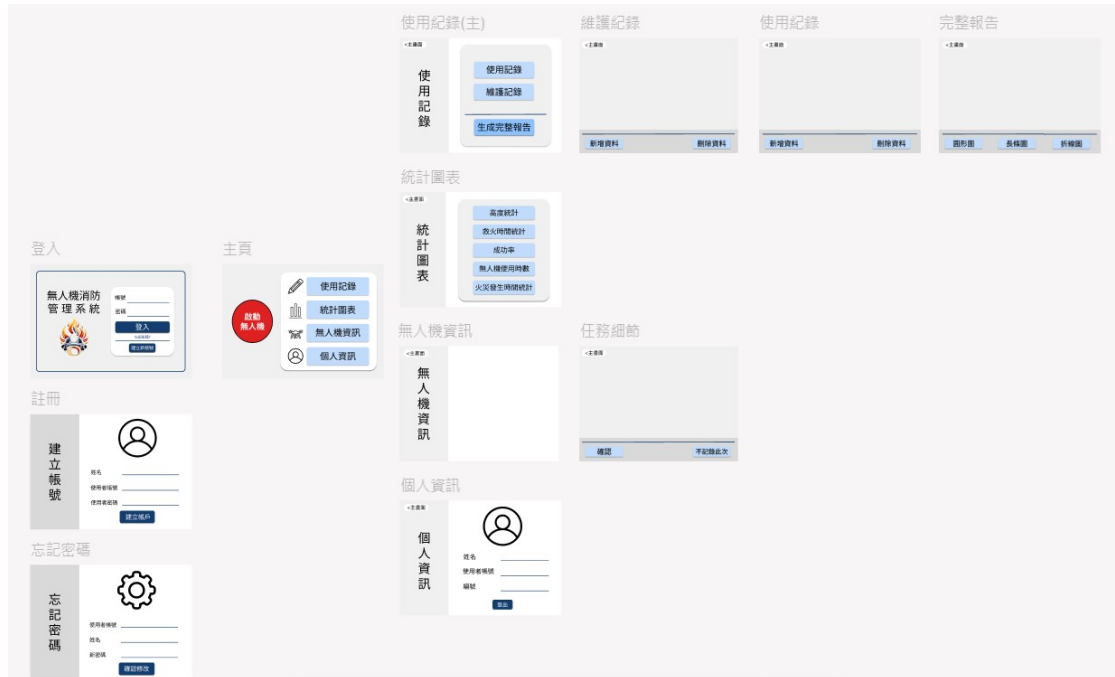
Users 表記錄用戶 ID(UserID), 用戶名(Username), 密碼(Password), 姓名(Name), 創建時間(CreateTime), 上次登錄時間>LastLoginTime)



	UserID	Username	Password	Name	CreationTime	LastLoginTime
1	1	user	123	aaa	2024-04-21 22:45:53	2024-08-07 20:57:25
2	2	admin	123	bbb	2024-07-30 15:49:42	2024-07-30 16:46:51
3	3	root	123	ccc	2024-07-30 15:54:12	2024-08-16 00:59:08
4	4	gggg	123	ddd	2024-07-30 17:45:19	2024-07-30 18:15:03
5	7	paris	12345	鄭哲斌	2024-08-07 21:51:17	2024-08-07 21:51:26

4. 使用者操作頁面(UI)及分析

使用 Figma 設計使用者介面，使介面簡潔且功能清晰。



- (1) 啟動無人機：紅色圓形按鈕，設計思維是希望以顯目的紅色，使得消防隊員能快速準確找到按鈕。按下此按鈕即可啟動無人機，使無人機進入救災模式。
- (2) 使用紀錄：內含救災使用紀錄、任務細節、維護紀錄，另可結合 Power BI 生成完整報告。
- (3) 統計圖表：結合 Python 生成統計圖表。
- (4) 無人機資訊：內含各台無人機資訊。
- (5) 個人資訊：包含消防隊員的基本資訊。

5. 前後端整合

前端使用 Figma 設計介面，用 Tkinter Designer 轉為 py 檔，介面中的各個元素會對應到操作，按下前端的按鈕時，觸發後端的邏輯程式，進而實現功能。

6. PowerBI 與後端的整合

PowerBI 連接 MySQL，並透過 <iframe> 的方式嵌入到報告中，進行數據可視化。跑 CreateReport.py 後，會在桌面創建一個.html，裡面匯出了結合 Power BI 分析的圖表及目前的使用記錄、維護記錄、無人機資訊等，無須安裝此系統即可查看資料。

```
gui > CreateReport.py
25 def generate_html_table(data, columns):
30
31 # 生成報告
32 def generate_report():
33     current_date = datetime.datetime.now().strftime("%Y-%m-%d")
34     queries = {
35         "無人機信息": "SELECT * FROM Drones",
36         "任務記錄": "SELECT * FROM Missions",
37         "維護記錄": "SELECT * FROM MaintenanceRecords"
38     }
39     columns = {
40         "無人機信息": ["無人機ID", "型號", "製造日期", "最大載重", "最大飛行時間", "最大飛行距離", "狀態"],
41         "任務記錄": ["任務ID", "無人機ID", "任務類型", "開始時間", "結束時間", "任務地點", "是否成功", "最大高度", "最低高度", "架次"],
42         "維護記錄": ["記錄ID", "無人機ID", "維護日期", "維護類型", "維護詳情", "維護費用"]
43     }
44
45     html_content = f"<h1>無人機協助消防系統分析報告 (產生日期: {current_date})</h1>"
46
47     # 加入 Power BI 圖表的 iframe 嵌入碼
48     powerbi_iframe = '<iframe title="0818" width="600" height="373.5" src="https://app.powerbi.com/view?r=eyJrIjoim2IwM2I0NTktYzlkZi00MWQ4LTk'
49     html_content += f"<h2>救災系統分析圖表</h2><{powerbi_iframe}>"
50
51     html_content += f"<h2>基礎資料信息</h2>"
52     for table_name, query in queries.items():
53         button_id = f"toggle_{table_name}"
54         html_content += f"<h3>{table_name}</h3><button onclick='document.getElementById('{button_id}').style.display = (document.getElementById("
55         data = fetch_data(query)
56         html_table = generate_html_table(data, columns[table_name])
57         html_content += f"<div id='{button_id}' style='display: none;'>{html_table}</div>"
58
59     timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
60     desktop_path = os.path.join(os.path.expanduser("~"), "Desktop")
61     filename = f"{desktop_path}\\分析報告_{timestamp}.html"
62
63     # 確保桌面路徑存在
```

7. Line Bot 可參考 p. 35

陸、系統開發環境及工具

一、系統使用環境介紹

1. 開發環境：

Windows、Linux

2. 使用工具：

PyCharm、MySQL、YOLOv8、

Power BI、Figma、LINE Developers、GitHub

3. 使用者安裝環境

OpenCV、Tkinter

4. 需要的程式語言：

Python

PyCharm	
MySQL	

YOLOv8	
Power BI	 Power BI
Figma	 Figma
LINE Developers	 LINE DC
GitHub	 GitHub
OpenCV	 OpenCV

Tkinter	
Python	

二、工具介紹

1. PyCharm:

PyCharm 是由 JetBrains 開發的一款強大且功能豐富的 Python 集成開發環境 (IDE)。它提供了廣泛的開發工具和功能，適合初學者和專業開發者使用。

2. MySQL:

MySQL 是一個開源的關係型數據庫管理系統 (RDBMS)，廣泛應用於各類應用程序中。它以高性能、高可靠性和便捷性著稱。

3. YOLOv8:

YOLOv8 (You Only Look Once, Version 8) 專為高效能實時影像處理而設計。它延續了YOLO架構的核心理念，以單次前向傳遞即可同時預測多個物體的邊界框和類別。相較於之前的版本，

具有更快的推理速度和更高的準確度，特別適合應用於需要低延遲的場景如自動駕駛、安防監控和機器人視覺。它透過引入新的架構優化技術，如改進的錨框設計和更深層的神經網路層次，進一步提高了模型的性能和適應性。

4. Power BI:

Power BI 是由微軟推出的商業智慧 (BI) 工具，用於資料視覺化和分析。它能輕鬆整合多種資料來源，並透過直觀的介面讓使用者創建互動式報表和儀表板。對於軟體工程師來說，Power BI 提供了強大的資料建模和DAX公式功能，能有效分析大規模資料集，並支援即時資料流的監控，適合應用於商業分析和決策支持系統。

5. Figma:

Figma是線上的介面設計工具，允許團隊進行多人協同合作，在瀏覽器中進行UI/UX設計，共同繪製使用者的介面原型，只要有一個成員變更內容，其他使用者都能即時查看並且變更或者加入編輯，使得團隊可以共同討論，創作出最有共識的產品介面。

6. LINE Developers:

LINE Developers 提供了一套 API 和 SDK，用於開發基於LINE 平台的應用和服務。它允許開發者集成消息、聊天機器人、

支付等功能到自己的應用中。

7. GitHub:

GitHub是一個基於Git的網路平台，用於程式碼儲存、版本控制和協作開發。它提供了一個集中化的存儲庫，讓開發者能夠上傳程式碼、追蹤更動、管理專案版本，並與其他開發者合作。

8. OpenCV:

OpenCV(Open Source Computer Vision Library)是一個開源的計算機視覺和影像處理庫。支持多種編程語言，廣泛應用於圖像處理、物體識別、機器學習等。

9. Tkinter:

Tkinter 是一個用於建立圖形使用者介面(GUI)的標準Python 函式庫。它提供了一組工具和小部件，用於建立具有互動式視窗、按鈕、選單、文字方塊和其他GUI元素的桌面應用程式。這是一個跨平台工具包，可建立在多個作業系統（包括Windows、macOS 和Linux）上執行的GUI應用程式。

10. Python:

Python 是一種高級、直譯式、通用的編程語言，因其簡單易學、功能強大而廣受歡迎。它適用於多種應用場景，如網頁開發、數據分析、人工智慧、科學計算等。

柒、結論及未來發展

一、結語

無人機在火災辨識中的應用已經證明其在提升反應速度、精確定位火源、減少救援風險與解放消防人力等各方面的巨大潛力。通過無人機的高空監控與數據實時傳輸，救援人員能更有效地應對火災，保障生命與財產安全。這一技術的成熟應用，不僅提高了現代消防的效率，也為消防救援行動帶來了新的突破。

二、未來展望

隨著無人機技術的持續進步，未來在火災辨識中將會有更高效、更智能的應用。例如，結合人工智慧和大數據分析，無人機可以實現自動火災預防與發生時即時撲滅火勢，並能夠在更複雜的環境中進行自主導航與任務執行。此外，無人機火災辨識技術有望擴展到更廣泛的城市防災與應急管理系統中，為全方位提升社會的災害應對能力提供強大助力。

捌、參考資料

1. <https://youtu.be/FBavXyN18K8>
2. <https://youtu.be/M5R9oHMI6A?si=TuaJUJMytuU-CBWO>
3. https://www.bilibili.com/video/BV1MN4y197Q3/?vd_source=0e86ab159b4b8ff7b8627641ada91b16
4. <https://www.youtube.com/watch?v=vrHz75qNQik>
5. https://blog.csdn.net/qq_42589613/article/details/134713170
6. https://blog.csdn.net/weixin_45277161/article/details/131046636
7. <https://blog.csdn.net/hfutdog/article/details/88085878>
8. <https://ithelp.ithome.com.tw/articles/10299634?sc=iThelpR>
9. <https://steam.oxxostudio.tw/category/python/ai/opencv-index.html>
10. <https://ithelp.ithome.com.tw/articles/10229943>
11. <https://steam.oxxostudio.tw/category/python/example/line-webhook.html#a1>
12. <https://ithelp.ithome.com.tw/articles/10229943>
13. <https://wenku.csdn.net/answer/7ibrd74us6>

玖、專題月進度表

一、110 下學期(發想階段，隔週開會)

2023.02 討論專題大方向：藉由分享資訊新聞(創意、內容)，

選組長：陳宇柔、副組長：曾紘杰。

2023.03 與清江老師開會討論。

2023.04 與培宇老師開會討論。

2023.05 與哲斌老師開會討論，決定專題教授。

2023.06 決定專題大方向為無人機應用。

二、111上學期(尋找題目及評估測試環境階段，隔週與老師開會和組內會)

2023.07 組員各自發想無人機應用，提出簡單功能介紹及找相關資料。

2023.08 每人繳交一份無人機的創意應用，並詳細列出功能及使用到的軟硬體工具。

2023.09 透過可行性分析和市場調查，組內投票選出兩個提案：1. 無人機室內設計3D建模 2. 無人機郊外巡邏。

2023.10 發現實作上有困難，聽取教授建議將題目更改為無人機火災辨識。

2023.11 架深度學習的環境，並學習相關知識。

2023.12 訓練模型成功，但準確率不足，提高訓練集火災相片張數，

增加至3萬張。技術改進部分：1. 檔案記錄改用logging套件 2. 火焰框會顯示火焰嚴重等級與次序 3. 記錄圖片並表示嚴重級別(20秒記錄一次或圖片過大)。

2024.01 各自提出十個功能，討論並製作功能架構圖(分為用戶端及管理端)。

三、111下學期(隔週與老師開會，每週組內會)

2024.02 建資料庫、製作要給消防員看的系統簡介影片、申請系上虛擬機。

2024.03 與新北市政府消防局第三大隊淡水消防分隊進行訪談及分享此系統進展。

2024.04 新增Line Bot系統(線上報警、防災知識連結等)。

2024.05 購買CoDrone EDU無人機(需求不符退貨)，資料庫需要新增(多台無人機、出動架次紀錄、出勤次數、火災統計、無人機統計、火災類型協助等。)

2024.06 使用Tkinter做簡易UI、尋找符合需求的無人機，最終購買Tello無人機。

2024.07 測試無人機程式控飛行，改進畫面傳輸進電腦的顏色問題。

2024.08 用Figma美化UI、製作現場演示的道具、搭建線上資料庫。

2024.09 最後測試 製作專題系統書。

拾、消防隊參訪



與新北市政府消防局
第三大隊淡水消防分
隊進行訪談



20240308消防隊訪談

1. 觀賞系統簡介影片

2. 台灣有沒有無人機的應用已經在實施或研發了？

目前用於消防都是情資搜集：火勢、範圍

Q：為何無法用來救災

A：沒有買到足夠的載具（承重、力道）

3. 如何判斷火災哪裡先救火（窗外）？

不建議從窗外射水，除非窗外射水柱，過渡射水（爭取民眾生存的時

間)，通常用水柱，水霧會讓空氣往前推，容易加助火勢成長。

4. 有沒有大樓失火的影片可以提供給我們？

5. 如果有無人機可以幫助救災 希望怎麼運用

(1)火災：無人機先噴泡沫（防止火焰變大），消防隊再噴水柱救災

(2)山難：找人

(3)水域救援：找人（水平面上用肉眼看不到）

6. 若推出一個消防員跟民眾用的系統，希望有什麼功能？

通知：安全區（熱區 有化學物質 民眾不可靠近）

7. 有什麼樣的資料對於消防是有作用的？

(1)轄區人口分佈、產業特色，會影響救災方式。

(2)有沒有救到人、滅火時間，可進行事後檢討

8. 如何判斷撲滅火焰優先順序、通常要噴幾秒（大小/顏色/煙霧）？

(1)初期：打火點，打到熄滅

(2)中期：打火點附近，用於降溫

(3)後期：火點已經控制，用排煙機/水霧排煙，由內往外打

9. 火警類型有哪些？

(1)住宅火警：通常不會從戶外射水，怕反而讓火蔓延，首要任務處理民眾受困，優先救人。

(2)廠房、農舍、化學工廠、槽車、裡面沒有人的時候。

10. 在現場什麼樣數據對於救火是有幫助的

溫度：看溫度高低 通常溫度高的地方是火點。

11. 救火順序是什麼？

(1)打119中心

(2)報案派遣

(3)確認報案人（看到狀況、有沒有人受困）

(4)通知附近消防隊到現場搜救（先進場救人）

12. 如何判斷火災原因？

(1)住宅火警：基本上都是電器火警

(2)工廠火警：確認安管系統，是否有危險物質 Ex. 易燃性、易爆炸

13. 什麼時候適合無人機救災？

(1)對人員有危害的地方

(2)初期到不了的地方

(3)沒有人受困的地方

(4)山難搜救

結論

1. 原先方式（窗外射水）不可實施，應改成噴泡沫較為恰當（輔助消防）

2. 規劃其它擴充功能

拾壹、分工表

職稱	姓名	負責部分
組長	陳宇柔	1. 使用者介面設計 2. 文件製作、排版
副組長	曾紘杰	1. 使用者介面與功能製作 2. Line Bot 伺服器 3. yolov8 火焰辨識模型訓練 4. 資料庫設計
組員	張楚霖	1. 無人機辨識 2. 無人機飛行 3. yolov8 火焰辨識模型展示介面
組員	呂紘燁	1. 上台報告
組員	朱婕瑜	1. 無人機辨識 2. 文件製作
組員	謝長恩	1. Power BI 分析圖表 2. 線上資料庫搭建 3. Line Bot 使用介面